

FROM RETRIEVAL AUGMENTED GENERATION TO ANALOGY AUGMENTED GENERATION

Anonymous authors

Paper under double-blind review

ABSTRACT

TODO

1 MOTIVATION

- Peng et al. (2023) demonstrate that augmenting with external knowledge sources and self-feedback improves performance on factual knowledge tasks.
- Huang et al. (2022) demonstrate that LLMs can produce procedural knowledge. But the VirtualHome environment is limited (and the choices of steps to include is subjective), and the paper does not leverage external knowledge (although it does leverage environment grounding).
- Factual knowledge is most succinctly represented by knowledge graphs, but these graphs leave out contextual information (which is why current approaches to factual generation use retrieved documents instead of KGs).
- Procedural knowledge can also be succinctly represented as a graph relating goals with steps, but this also leaves out contextual information.
- Retrieving procedural knowledge can be difficult because information is often spread across many more documents.
 - To reliably discover the information the model needs, it is already established practice to threshold retrieved documents based on embedding similarity with the query, or to prompt an LLM to filter or compress the passages.
- What is procedural knowledge: does pseudocode count? does instructions to get to the university count? what makes it procedural?
 - Procedural knowledge is action knowledge.
 - Procedural knowledge comes from *motivated* reasoning over factual knowledge.
 - Humans tend to construct reusable chunks of actions, and recalling and modifying them as needed (see here).
- What we’re doing is different from general summarization because the relevance of individual facts depends on the goal being achieved. It’s also different from knowledge extraction because the resulting procedure has to cohesively combine many facts to produce steps, and the end result won’t necessarily spell out the underlying facts.
- For performance on procedural knowledge, the constraint is procedural reasoning, not fact retrieval. (This is demonstrated by the performance of GPT-4 when just doing retrieval.)

2 LCSTEP

- talk about procedural knowledge design principles
- explain what a good procedure looks like (main idea steps, no unnecessary details, etc.) so others can follow (regardless of the schema they choose for their data)
- Our dataset is a procedural knowledge base for using langchain.
- The primary task will be procedure generation, and the downstream tasks will be code generation and (as a bonus) code writing Q&A.

- Each procedure also comes with unit tests that check whether generated code accomplishes the procedure.

LCStep contains three sets of documents: API reference, conceptual documentation, and procedures. See Figure 1 for a diagram of the process of generating the LCStep data.

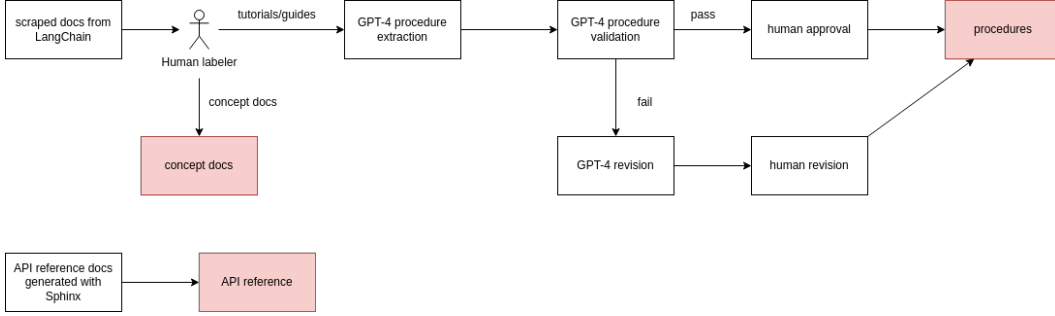


Figure 1: The workflow used to generate the LCStep dataset.

2.1 API REFERENCE

We generate the API reference material from the source files in the LangChain GitHub repository using Sphinx. These files contain descriptions of all APIs in the Python package, including call signatures and argument descriptions. These files do not contain any usage examples or high-level explanation.

2.2 CONCEPTUAL DOCUMENTATION AND PROCEDURES

We collect these documents by scraping the LangChain Python docs. We manually filter out topic pages and stubs, leaving 228 documents. We then manually classified these into around 30 documents of conceptual documentation, and around 180 documents containing tutorials/guides.

For the 180 tutorials/guides, we prompted GPT-4¹ to extract a list of high-level steps necessary to accomplish the goal. We then prompted GPT-4² to rate those extracted procedures using a list of criteria. We found that this caught many mistakes where GPT-4 did not follow all the stated instructions. In those cases, we had the model revise the steps to meet the requirements, and then we manually checked the revised versions.

3 ANALOGY-AUGMENTED GENERATION

The performance of retrieval-augmented generation (RAG) is highly dependent on the number and size of the documents, as well as the density of relevant information (citation needed). For many tasks, a lot of the knowledge to be retrieved is the same for many queries. In our case, many procedures involving the LangChain library require connecting to a language model, creating a prompt template, and constructing a chain. Instead of collecting this information anew for each similar query, we search over past outputs to find procedures similar to the current query, and then include them in the set of retrieved candidate documents to include in the final prompt.

A good procedural knowledge representation provides context in natural language. This makes the individual parts composable.

- **Bootstrap skills:** We first bootstrap the skill library by asking the model to create a few procedures from the concept documents. This will include simple tasks like “call an OpenAI language model”.

¹See Appendix ?? for the full prompt.

²See Appendix ?? for the full prompt.

- **Search skill library:** When we get a new task to generate a procedure for, we first search for related tasks in the skill library to produce a candidate procedure. We then ask the model if any of these are close enough to start working with. We then prompt the model to point out places where the procedure should be different, and mark those with e.g. (TODO change this to a Cohere model instead of OpenAI).
- **If nothing similar enough, search docs:** If none were similar, we search over the concept docs & API reference using regular retrieval, so the model has an idea of what broad steps it needs to take. The prompt might look like “Here are some documents describing basic actions with LangChain. {...} Generate a candidate procedure for the task ‘{...}’ based on the conceptual documentation above. If you don’t have enough information to fill in the details for a part, write (TODO add more detail) at the beginning of that part”.
- **Iteratively update & improve:** We now have a candidate procedure, marked with TODOs where things need to be changed or become more specific. We then handle those TODOs one at a time: we create a retrieval query to search the skill library, API ref, and concept docs for that specific step & TODO. We then provide that info to the model and ask it to modify the procedure to resolve the TODO. We do this until the TODOs are removed.
- **Review & compare with requirements:** We then allow the model another chance to change the generated procedure by asking it if it meets the requirements we’ve defined for procedures. This might allow it to go back and make certain sections more specific.
- **Store and return:** We then store the generated procedure in the skill library, and return the procedure as the answer for the how to do the given task.
 - When storing, we ask the model to compare the procedure with others in the skill library. If the procedure is relatively distinct, we store it. If not, we ask the model to combine it with the existing one to make it more general. This may involve splitting out steps into sub-goals, which will allow those higher steps to be general (thus reducing the number of different steps in the skill library).

This is like storing experience (i.e. how to do something) instead of facts (i.e. what something is) in memory. It’s also like learning to use tools because we collect a library of “skills”, but it’s different because we don’t treat the skills as black boxes: we use them to draw analogies with to solve novel problems.³ We can call this “analogy-augmented generation” (AAG).

grounding stuff:

- This approach can still benefit from environment grounding much like Voyager or the two papers cited at the top.
- In our case, we ground the output by transforming it into code and then evaluating the code. The first test is whether the code runs; the next is whether the LangChain model/chain/agent produces results that look right to a human or LLM, when compared with the examples from the tutorials.
- This method works anywhere real, simulated, or user feedback can be provided. (Component should be generic enough to get feedback from any source. For example, some procedure might need a Cohere model, which we won’t have a token for. This will be for a second paper.)

4 EXPERIMENTS

- understand how well plain RAG can produce procedural knowledge
 - model: GPT-4
 - compare model’s output on a few examples with and without the procedural knowledge base, by hand
 - highlight examples where procedural reasoning falls short with just with plain retrieval

³See the Wikipedia articles for procedural knowledge and procedural memory.

- test whether compressing the documents helps RAG (It’s likely that the standard model gets lost in long context and needs more compressed content.)
- full procedural knowledge generation test
 - models: GPT-3.5/4, Llama2 instruction-tuned (both prompted and fine-tuned [with PEFT?] versions of each) (plain, normal compression, and AAG versions)
 - evals:
 - * ROUGE
 - * self-check GPT
 - * GPT-4-rated relevancy, faithfulness, context recall, correctness
 - TODO borrow existing metrics, like OwlEval, or page 39 of FLASK
 - select a set of hardest procedures and report performance on that set
 - find examples where the model (un)successfully made analogy to produce the right procedure
 - Questions we want to answer with the RAG vs. fine-tuning experiments:
 - * How do we draw the line between using RAG and fine-tuning, based on the size and label types and publicness of the dataset?
- TODO experiment with token limits?
 - Set smaller token limits artificially for 100k models, and do the same for llama2 (the OSS standard).
- code generation
 - models: same as above, but we post-process the generated procedures to create code
 - evals:
 - * does the code run?
 - * “unit tests” collected from LCStep tutorials, as described in the methods section
 - TODO think of a mechanism for feedback from code all the way to the skill library
- TODO code-writing question answering
 - get high-quality questions from forums, and then have lab members judge quality of SO-style answers (see SO tag for LangChain or LangChain OpenAI Qs)

To demonstrate that AAG can overcome shortcomings of standard retrieval, we pit the two methods against each other on the task of generating the procedures in LCStep. We also test against a few baselines and oracles which we explain below. Both methods have access to the API reference and conceptual documentation from the dataset.

4.1 BASELINES

All prompting methods are performed with GPT-4-0614 from OpenAI as well as an instruction-tuned version of LLaMA2.

- The **few-shot** approach will prompt the model with a basic instruction and a few fixed examples of the output format. We expect this method to fail due to a lack of knowledge of LangChain in the models’ training data.
- The vanilla retrieval-augmented approach (**RAG**) constructs a prompt by doing retrieval over a vector store containing the conceptual documentation and API reference material.
 - We expect to further refine this retrieval process somewhat. This may involve enforcing retrieval diversity by penalizing similarity between retrieved docs, having the model split the query into subqueries to find the individual parts necessary to build the whole answer, or using contextual compression.
 - This method would also benefit from HyDE (Gao et al., 2022), I³ (Dong et al., 2023), or EAR (Chuang et al., 2023). At that point, it might be worth having this optimized version as a separate baseline here.

For our experiment, the **AAG** approach constructs a prompt by retrieval over the same documents as RAG, plus all procedures generated so far. When retrieving from the past outputs, the model is separately presented with the selection prompt as described in Section ??.

4.2 EXTRA

- demo UI to demonstrate procedural knowledge-enhanced code copilot (more efficient? stronger reasoning? experience accumulation? better task solving strategy? better explainability?)

REFERENCES

- Yung-Sung Chuang, Wei Fang, Shang-Wen Li, Wen tau Yih, and James R. Glass. Expand, rerank, and retrieve: Query reranking for open-domain question answering. *Annual Meeting Of The Association For Computational Linguistics*, 2023. doi: 10.48550/arXiv.2305.17080.
- Qian Dong, Yiding Liu, Qingyao Ai, Haitao Li, Shuaiqiang Wang, Yiqun Liu, Dawei Yin, and Shaoping Ma. I³ retriever: Incorporating implicit interaction in pre-trained language models for passage retrieval. *arXiv preprint arXiv: 2306.02371*, 2023.
- Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. *Annual Meeting Of The Association For Computational Linguistics*, 2022. doi: 10.48550/arXiv.2212.10496.
- Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato (eds.), *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 9118–9147. PMLR, 17–23 Jul 2022. URL <https://proceedings.mlr.press/v162/huang22a.html>.
- Baolin Peng, Michel Galley, Pengcheng He, Hao Cheng, Yujia Xie, Yu Hu, Qiuyuan Huang, Lars Liden, Zhou Yu, Weizhu Chen, and Jianfeng Gao. Check your facts and try again: Improving large language models with external knowledge and automated feedback. *arXiv preprint arXiv: 2302.12813*, 2023.